

Regular expressions in grep (regex) with examples

Author: Vivek Gite • Last updated: September 6, 2023 • [83 comments](#)

How do I use the [grep command](#) with regular expressions on a Linux and Unix-like operating systems? How do I use grep and regular expressions (regex) to search for text/ words in Linux?



Linux comes with GNU grep, which supports extended regular expressions. GNU grep is the default on all Linux systems. The [grep command](#) is used to locate information stored anywhere on your server or workstation. Let us see fundamental of regex and how to use regular expressions in the Linux and Unix like systems.

Tutorial details	
Difficulty level	Easy .
Root privileges	No
Requirements	Linux or Unix terminal
Category	Searching
Prerequisites	egrep command
OS compatibility	BSD • Linux • macOS • Unix • WSL
Est. reading time	7 minutes

nixCraft: Privacy First, Reader Supported

- **nixCraft is a one-person operation.** I create all the content myself, with no help from AI or ML. I keep the content accurate and up-to-date.
- **Your privacy is my top priority.** I don't track you, show you ads, or spam you with emails. Just pure content in the true spirit of Linux and FLOSS.
- **Fast and clean browsing experience.** nixCraft is designed to be fast and easy to use. You won't have to deal with pop-ups, ads, cookie banners, or other distractions.
- **Support independent content creators.** nixCraft is a labor of love, and it's only possible thanks to the support of our readers. If you enjoy the content, please support us on Patreon or share this page on social media or your blog. Every bit helps.

[Join Patreon →](#)

Regular Expressions in grep

Regular Expressions is nothing but a pattern to match for each input line. A pattern is a sequence of characters. Following all are examples of pattern:

```
^w1
w1|w2
[ ^ ]
foo
bar
[0-9]
```

Three types of regex

The grep understands three different types of regular expression syntax as follows:

1. basic (BRE)

2. extended (ERE)

3. perl (PCRE)

grep Regular Expressions Examples

Search for a word named 'vivek' in the [/etc/passwd](#) file:

```
$ grep 'vivek' /etc/passwd
```

Sample outputs:

```
vivek:x:1000:1000:Vivek Gite,,,:/home/vivek:/bin/bash
vivekgite:x:1001:1001::/home/vivekgite:/bin/sh
gitevivek:x:1002:1002::/home/gitevivek:/bin/sh
```

Next, search for a word named 'vivek' in any case (i.e. case insensitive search):

```
$ grep -i -w 'vivek' /etc/passwd
```

Let us try to search two words 'vivek' or 'raj' in any case:

```
$ grep -E -i -w 'vivek|raj' /etc/passwd
```

The PATTERN in last example, used as an extended regular expression. The following will match word Linux or UNIX in any case using the [egrep command](#):

```
$ egrep -i '^(linux|unix)' filename
# Same as above by passing the '-E' to the grep #
$ grep -E -i '^(linux|unix)' filename
```

A note about `egrep` vs. `grep -E` syntax

The latest version of egrep will show the following warning

```
egrep: warning: egrep is obsolescent; using grep -E
Usage: grep [OPTION]... PATTERNS [FILE]...
Try 'grep --help' for more information.
```

You need to update all your scripts and command to use the following syntax. From:

```
$ egrep -i 'foo|bar' /path/to/file
```

To (avoid using the `egrep`):

```
$ grep -E -i 'foo|bar' /path/to/file
```

How to match single characters

The `.` character (period, or dot) matches any one character. Consider the following demo.txt file:

```
$ cat demo.txt
```

Sample outputs:

```
foo.txt
bar.txt
foo1.txt
bar1.doc
foobar.txt
foo.doc
bar.doc
```

```
dataset.txt  
purchase.db  
purchase1.db  
purchase2.db  
purchase3.db  
purchase.idx  
foo2.txt  
bar.txt
```

Let us find all filenames starting with purchase, type:

```
$ grep 'purchase' demo.txt
```

Next I need to find all filenames starting with purchase and followed by another character:

```
$ grep 'purchase.db' demo.txt
```

Our final example find all filenames starting with purchase but ending with db:

```
$ grep 'purchase..db' demo.txt
```

```
vivek@nixcraft-asus:/tmp$ cat demo.txt
List of files:
foo.txt
bar.txt
foo1.txt
bar1.doc
foobar.txt
foo.doc
bar.doc
dataset.txt
purchase.db
purchase1.db
purchase2.db
purchase3.db
purchase.idx
foo2.txt
bar.txt
vivek@nixcraft-asus:/tmp$ grep 'purchase' demo.txt
purchase.db
purchase1.db
purchase2.db
purchase3.db
purchase.idx
vivek@nixcraft-asus:/tmp$ grep 'purchase.' demo.txt
purchase.db
purchase1.db
purchase2.db
purchase3.db
purchase.idx
vivek@nixcraft-asus:/tmp$ grep 'purchase.db' demo.txt
purchase.db
vivek@nixcraft-asus:/tmp$ grep 'purchase..db' demo.txt
purchase1.db
purchase2.db
purchase3.db
vivek@nixcraft-asus:/tmp$
```

© www.cyberciti.biz

How to match only dot (.)

A dot (.) has a special meaning in regex, i.e. match any character. But, what if you need to match dot (.) only? I want to tell my grep command that I want actual dot (.) character and not the regex special meaning of the . (dot) character. You can escape the dot (.) by preceding it with a \ (backslash):

```
$ grep 'purchase..' demo.txt
$ grep 'purchase.\.' demo.txt
```

```
vivek@nixcraft-asus:/tmp$ grep 'purchase..' demo.txt
purchase.db
purchase1.db
purchase2.db
purchase3.db
purchase.idx
vivek@nixcraft-asus:/tmp$ grep 'purchase.\.' demo.txt
purchase1.db
purchase2.db
purchase3.db
vivek@nixcraft-asus:/tmp$
```

© www.cyberciti.biz

Anchors

You can use ^ and \$ to force a regex to match only at the start or end of a line, respectively. The following example displays lines starting with the vivek only:

```
$ grep ^vivek /etc/passwd
```

Sample outputs:

```
vivek:x:1000:1000:Vivek Gite,,,:/home/vivek:/bin/bash
vivekgite:x:1001:1001::/home/vivekgite:/bin/sh
```

You can display only lines starting with the word vivek only i.e. do not display vivekgite, vivekg etc:

```
$ grep -w ^vivek /etc/passwd
```

Find lines ending with word foo:

```
$ grep 'foo$' filename
```

Match line only containing foo:

```
$ grep '^foo$' filename
```

You can search for blank lines with the following examples:

```
$ grep '^$' filename
```

Matching Sets of Characters

How to match sets of character using grep

The dot (.) matches any single character. You can match specific characters and character ranges using [..] syntax. Say you want to Match both 'Vivek' or 'vivek':

```
$ grep '[vV]ivek' filename
```

OR

```
$ grep '[vV][iI][Vv][Ee][kK]' filename
```

Let us match digits and upper and lower case characters. For example, try to match words such as vivek1, Vivek2 and so on:

```
$ grep -w '[vV]ivek[0-9]' filename
```

In this example match two numeric digits. In other words match foo11, foo12, foo22 and so on, enter:

```
$ grep 'foo[0-9][0-9]' filename
```

You are not limited to digits, you can match at least one letter:

```
$ grep '[A-Za-z]' filename
```

Display all the lines containing either a “w” or “n” character:

```
$ grep [wn] filename
```

Within a bracket expression, the name of a character class enclosed in “[:]” and “[:]” stands for the list of all characters belonging to that class. Standard character class names are:

- `[:alnum:]` – Alphanumeric characters.
- `[:alpha:]` – Alphabetic characters
- `[:blank:]` – Blank characters: space and tab.
- `[:digit:]` – Digits: ‘0 1 2 3 4 5 6 7 8 9’.
- `[:lower:]` – Lower-case letters: ‘a b c d e f g h i j k l m n o p q r s t u v w x y z’.
- `[:space:]` – Space characters: tab, newline, vertical tab, form feed, carriage return, and space.
- `[:upper:]` – Upper-case letters: ‘A B C D E F G H I J K L M N O P Q R S T U V W X Y Z’.

In this example match all upper case letters:

```
$ grep '[:upper:]' filename
```

How negates matching in sets

The ^ negates all ranges in a set:

```
$ grep '[vV]ivek[^0-9]' test
```



```
vivek@nixcraft-asus:/tmp$ cat test
My name it Vivek Gite.
vivek loves Unix and Linux too.
My Linux user name is vivek and password is *****
My CentOS user name is vivek1 and password is *****
vivek@nixcraft-asus:/tmp$ grep '[vV]ivek' test
My name it Vivek Gite.
vivek loves Unix and Linux too.
My Linux user name is vivek and password is *****
My CentOS user name is vivek1 and password is *****
vivek@nixcraft-asus:/tmp$ grep '[vV]ivek[^0-9]' test
My name it Vivek Gite.
vivek loves Unix and Linux too.
My Linux user name is vivek and password is *****
vivek@nixcraft-asus:/tmp$
```

© www.cyberciti.biz

Using grep regular expressions to search for text patterns

Wildcards

You can use the “.” for a single character match. In this example match all 3 character word starting with “b” and ending in “t”:

```
grep '\<b.t\>' filename
```

Where,

- \< Match the empty string at the beginning of word
- \> Match the empty string at the end of word.

Print all lines with exactly two characters:

```
$ grep '^..$' filename
```

Display any lines starting with a dot and digit:

```
$ grep '^\. [0-9]' filename
```

Escaping the dot

Say you just want to match an IP address 192.168.2.254 and nothing else. The following regex to find an IP address 192.168.1.254 will not work (remember the dot matches any single character?):

```
$ grep '192.168.1.254' hosts
```

Sample outputs:

```
192.168.2.18      centos7
192x168y2z18     centos7
```

All three dots need to be escaped:

```
192.168.2.18      centos7
```

```
$ grep '192\.168\.1\.254' hosts
```

The following example will only match an IP address:

```
$ grep -E '[[[:digit:]]{1,3}\. [[[:digit:]]{1,3}\. [[[:digit:]]{1,3}\. [[[:digit:]]{1,3}]' file
```

How Do I Search a Pattern Which Has a Leading Symbol?

Searches for all lines matching ' option Without , grep would attempt to parse '' as a list of options. For example:

```
$ grep -e '--test--' filename
```

How Do I do OR with grep?

Use the following syntax:

```
$ grep -E 'word1|word2' filename
### OR ###
$ egrep 'word1|word2' filename
```

OR Try the following syntax:

```
$ grep 'word1\|word2' filename
```

How do I AND with grep?

Use the following syntax to display all lines that contain both 'word1' and 'word2'

```
$ grep 'word1' filename | grep 'word2'
```

OR try the following syntax:

```
$ grep 'foo.*bar\|word3.*word4' filename
```

How Do I Test Sequence?

You can test how often a character must be repeated in sequence using the following syntax:

```
{N}  
{N,}  
{min,max}
```

Match a character "v" two times:

```
$ egrep "v{2}" filename
```

The following will match both "col" and "cool" words:

```
$ egrep 'co{1,2}l' filename
```

Our next example will match any row of at least three letters 'c'.

```
$ egrep 'c{3,}' filename
```

In this example, I will match mobile number which is in the following format 91-1234567890 (i.e TwoDigit-TenDigit)

```
$ grep "[[:digit:]]\{2\}[-]\?[[[:digit:]]\{10\}" filename
```

How Do I Highlight with grep?

Pass the `--color` as follows:

```
$ grep --color regex filename
```

How Do I Show Only The Matches, Not The Lines?

Use the following syntax:

```
$ grep -o regex filename
```

grep Regular Expression Operator

I hope following table will help you quickly understand regular expressions in grep when using under Linux or Unix-like systems:

Table 1: grep (egrep) regex operator

Operator	Description	Ex

Operator	Description	Example
.	Matches any single character.	<pre>grep '.' file grep 'foo.' input</pre>
?	The preceding item is optional and will be matched , at most, once.	<pre>grep 'vivek?' /etc/passwd</pre>
*	The preceding item will be matched zero or more times.	<pre>grep 'vivek*' /etc/passwd</pre>
+	The preceding item will be matched one or more times.	<pre>ls /var/log/ grep +</pre>
{N}	The preceding item is matched exactly N times.	<pre>egrep '[0-9]{2}' /etc/passwd</pre>
{N, }	The preceding item is matched N or more times.	<pre>egrep '[0-9]{2,}' /etc/passwd</pre>
{N, M}	The preceding item is matched at least N times, but not more than M times.	<pre>egrep '[0-9]{2,4}' /etc/passwd</pre>
-	Represents the range if it's not first or last in a list or the ending point of a range in a list.	<pre>grep ':/bin/[a-z-]' /etc/passwd</pre>
^	Matches the empty string at the beginning of a line; also represents the characters not in the range of a list.	<pre>grep '^vivek' /etc/passwd grep '^[0-9]*' /etc/passwd</pre>
\$	Matches the empty string at the end of a line.	<pre>grep '^\$' /etc/passwd</pre>
\b	Matches the empty string at the edge of a word.	<pre>grep '\bvivek' /etc/passwd</pre>

Operator	Description	Example
<code>\B</code>	Matches the empty string provided it's not at the edge of a word.	<code>grep '\B/bin/bas'</code>
<code>\<</code>	Match the empty string at the beginning of word.	<code>grep '\<vivek' /c</code>
<code>\></code>	Match the empty string at the end of word.	<code>grep 'bash\>' /e</code> <code>grep '\<vivek\>'</code>

Linux grep vs egrep command

The egrep is the same as `grep -E` command. It interpret PATTERN as an extended regular expression. From the grep man page:

In basic regular expressions the meta-characters `?`, `+`, `{`, `|`, `(`, `)`, `\|`, `\(`, and `\)`.

Traditional egrep did not support the `{` meta-character, and some grep -E patterns and should use `[{]` to match a literal `{`.

GNU grep -E attempts to support traditional usage by assuming the For example, the command `grep -E '{1}` searches for the two-chara POSIX.2 allows this behavior as an extension, but portable scrip

Conclusion

You learned how to regular expressions (regex) in grep running on Linux or Unix with various examples. See GNU/grep man page online [here](#) or see the following resources using the [man command](#) or info command (or pass the [--help option](#):

```
$ man egrep
$ info egrep
$ man 7 regex
$ egrep --help
```

This entry is **2 of 7** in the **Linux / UNIX grep Command Tutorial** series. Keep reading the rest of the series:

1. [How To Use grep Command In Linux / UNIX](#)
2. Regular Expressions In grep
3. [Search Multiple Words / String Pattern Using grep Command](#)
4. [Grep Count Lines If a String / Word Matches](#)
5. [Grep From Files and Display the File Name](#)
6. [How To Find Files by Content Under UNIX](#)
7. [grep command: View Only Configuration File Directives](#)

Did you notice? 😊

nixCraft is ad-free to protect your privacy and security. We rely on reader support to keep the site running. Please consider subscribing to us on Patreon or supporting us with a one-time support through PayPal or purchase official merchandise. Your support will help us cover the costs of hosting, CDN, DNS, and tutorial creation.

[Patreon](#) →

[PayPal](#) →

[Shop](#) →

About the author: Vivek Gite is the founder of nixCraft, the oldest running blog about Linux and open source. He wrote more than 7k+ posts and helped numerous readers to master IT topics. Join the nixCraft community via [RSS Feed](#) or [Email Newsletter](#).

 Was this helpful? Please add [a comment to show your appreciation or feedback](#).

 To search, type & hit enter...

Related Tutorials

83 comments... [add one](#) ↓

Shivam Garg

Feb 16, 2010 @ 11:21

if you want know the line number of found match so you can use -n attributes.

cmd: `grep -n printf *.c`

This will show you all printf in c files with line number.

Some time we need the result in reverse manner. like i want to search all line that don't have 'printf'.

cmd: `grep -v printf *.c`

this will show all line that don't have printf.

HAPPY PROGRAMMING !!!!!!!!

↩ ∞

marc

Feb 16, 2010 @ 11:42

thx for the regex examples. verry usefull

↩ ∞

Zdenek Styblik

Feb 16, 2010 @ 11:43

there is also `% grep -F;`, formerly known as `% fgrep;`, which provides fixed string matching and is faster.

by man page, use of `% fgrep;` and `% egrep;` is deprecated and `% grep -F;` and `% grep -E;` should be used instead.

great summary!

↩ ∞

Dave

Feb 16, 2010 @ 15:16

Ok If i am tailing a firewall log with

`tail -F /log/myfirewall.log |grep -i 135`

I get results for port 135 but also 1352 for example, how do i use grep to only display port 135 and not 1352.

↩ ∞

🛡 **nixCraft**

Feb 16, 2010 @ 17:15

```
tail -f /log/myfirewall.log |grep -w '135'
```

↩ ∞

Hossam Abdelmonem

Feb 17, 2010 @ 4:30

Many Thanks Vivek for your great post, but let me correct on command with grep using wildcards, you typed :

grep '^.[0-9]' filename

Display any lines starting with a dot and digit, but this is wrong, and the right as the following:

grep -E '^\.|[0-9]' wildcards.txt

Thanks,

↩ ∞

buckCherry

Sep 10, 2012 @ 6:21

The above example "grep -E '^\.|[0-9]' wildcards.txt" is not also correct. This will match "a9b" which should not be matched.

The correct expression is: grep -E "^\.[0-9]" wildcards.txt

Note: the caret '^' when appear at the beginning indicates a line start anchor. However this is not all. Due to the OR '|' symbol in this case, a line can start matching with "[0-9]" and to ensure that all lines that doesn't start with 'dot' when takes alternate path must ensure that it starts with only digit, we need to prefix another '^' symbol.

Tim Boyer

Feb 18, 2010 @ 4:03

Only thing I miss from other Unices is grepping for a metacharacter. For instance, in dg/ux to count the number of tabs in a document I could do a

```
grep -c \t
```

or a \n for newlines, a \f for page feeds, etc.

Apparently, this doesn't work in Linux – I've changed those scripts to perl scripts.

David Malouf

Jan 18, 2012 @ 18:37

To use Tabs, use \t as expected followed by a qualifier (ex. *, +, ?)

For example:

```
grep -e '\t?'
```

Will find 1 or No Tabs. \t* will find 0 or more Tabs.

Although I must say, this comment thread got me thinking to add the qualifier. Thanks to all who post ideas, questions, etc. so the rest of us can learn!!

David

Tim Boyer

Jan 19, 2012 @ 1:09

Unfortunately, that seems not to work – at least in RHEL5

```
[tim@kyushu ~]$ cat testgrep
```

```
Test
```

```
T est
```

```
Test 1
```

```
T e s t
```

```
notatest
```

```
test 1
```

(All of those whitespaces are tabs)

```
[tim@kyushu ~]$ grep -e '\t?' testgrep
```

```
[tim@kyushu ~]$
```

← ∞

David Malouf

Jan 24, 2012 @ 1:02

Maybe is upper-case 'E' ? Just a shot-in-the-dark.

David

← ∞

Tim Boyer

Jan 24, 2012 @ 1:12

-E returns... everything. Including the lines that absolutely have no tab in them.

```
[tim@kyushu ~]$ cat testgrep
Test
T est
Test 1
T e s t
notatest
test 1
thereisnotabhere
[tim@kyushu ~]$ grep -E '\t?' testgrep
Test
T est
Test 1
T e s t
notatest
test 1
thereisnotabhere
[tim@kyushu ~]$
```

← ∞

jcxz100

Oct 28, 2014 @ 13:20

I have found a solution (see end of post).

—

But I can't do a simple grep for TABs either.

I did find out what's wrong when, above, all lines are returned: That's because your (and my) grep doesn't understand the '\t' – therefore it ignores the '\' part of the regex string and goes on to match any lines with lowercase 't' in it – unfortunately, in your cases, that means *every* single

line, because you didn't enter any line without a lowercase 't' ;-)
My grep doesn't understand hex, octal or unicode ('\\xFF', '77', or \\uFFFF) sequences either.

The only whitespace marker that works with my grep is '\\s', and that matches all types of blank: ' ', TAB, FF, and (when newlines are treated as ordinary characters) CR, and LF.

—

My test file looks like this:

```
lsb@lsb-t61-mint ~ $ cat testgrep-tabs.txt
1.notamatch
2.TabTest-no-tabs-here
3.a-line-which-will-always-be-skipped
4.TABT EST
5.TabTest 1
6.tab test 2
7.T a b T e s t
8.this line only has ordinary spaces (ascii 32 = hex 20)
9.first there are ordinary spaces, but now: a TAB
10.ignored-line
lsb@lsb-t61-mint ~ $
```

(Except for line 8 and 9, all lines that appear to have ordinary space(s) in them do in fact have TAB(s).

Line 9 has mostly ordinary spaces, but between the words 'now:' and 'a' is a single TAB char.)

And now for my examples. They are grouped for not repeating a lot of identical print outs.

—

The following commands do exactly the same: They print every line with a lowercase 't' in it:

```
(A1) lsb@lsb-t61-mint ~ $ grep '\t' testgrep-tabs.txt
(A2) lsb@lsb-t61-mint ~ $ grep -e '\t' testgrep-tabs.txt
(A3) lsb@lsb-t61-mint ~ $ grep -E '\t' testgrep-tabs.txt
(B1) lsb@lsb-t61-mint ~ $ grep '[\t]' testgrep-tabs.txt
(B2) lsb@lsb-t61-mint ~ $ grep -e '[\t]' testgrep-tabs.txt
(B3) lsb@lsb-t61-mint ~ $ grep -E '[\t]' testgrep-tabs.txt
(C1) lsb@lsb-t61-mint ~ $ grep '[\t]+' testgrep-tabs.txt
(C2) lsb@lsb-t61-mint ~ $ grep -e '[\t]+' testgrep-tabs.txt
(C3) lsb@lsb-t61-mint ~ $ grep -E '[\t]+' testgrep-tabs.txt
(D1) lsb@lsb-t61-mint ~ $ grep '[\t]{1,}' testgrep-tabs.txt
(D2) lsb@lsb-t61-mint ~ $ grep -e '[\t]{1,}' testgrep-tabs.txt
(D3) lsb@lsb-t61-mint ~ $ grep -E '[\t]{1,}' testgrep-tabs.txt
(E1) lsb@lsb-t61-mint ~ $ grep 't?' testgrep-tabs.txt
(E2) lsb@lsb-t61-mint ~ $ grep -e 't?' testgrep-tabs.txt
(E3) lsb@lsb-t61-mint ~ $ grep -E 't?' testgrep-tabs.txt
```

1.notamatch

2.TabTest-no-tabs-here

5.TabTest 1

6.tab test 2

7.T a b T e s t

8.this line only has ordinary spaces (ascii 32 = hex 20)

9.first there are ordinary spaces, but now: a TAB

```
lsb@lsb-t61-mint ~ $
```

It makes no difference whether I use double- or single-quotes around the regex string.

Note: I included the regex 't?' even though it is a little incorrect; because – if

it worked – it would simply match the sequence “a TAB char that may be followed by another char”.

—

The following commands produce no output at all (even though TAB is hex 9 = oct 011):

```
(A1) lsb@lsb-t61-mint ~ $ grep '\x09' testgrep-tabs.txt
(A2) lsb@lsb-t61-mint ~ $ grep -e '\x09' testgrep-tabs.txt
(A3) lsb@lsb-t61-mint ~ $ grep -E '\x09' testgrep-tabs.txt
(B1) lsb@lsb-t61-mint ~ $ grep '11' testgrep-tabs.txt
(B2) lsb@lsb-t61-mint ~ $ grep -e '11' testgrep-tabs.txt
(B3) lsb@lsb-t61-mint ~ $ grep -E '11' testgrep-tabs.txt
lsb@lsb-t61-mint ~ $
```

—

These commands match and print all the lines that have some kind of whitespace in them:

```
(A1) lsb@lsb-t61-mint ~ $ grep '\s' testgrep-tabs.txt
(A2) lsb@lsb-t61-mint ~ $ grep -e '\s' testgrep-tabs.txt
(A3) lsb@lsb-t61-mint ~ $ grep -E '\s' testgrep-tabs.txt
4.TABT EST
5.TabTest 1
6.tab test 2
7.T a b T e s t
8.this line only has ordinary spaces (ascii 32 = hex 20)
9.first there are ordinary spaces, but now: a TAB
lsb@lsb-t61-mint ~ $
```

That is a bit much; but it leads to the next portion:

—

WHAT WORKS

Following command is quite complex to look upon, but it works (at least for me it does):

```
lsb@lsb-t61-mint ~ $ grep '\s' testgrep-tabs.txt | sed -z -E 's/[\n|^][^\t]*[\n|$]/\n/g'
```

4.TABT EST

5.TabTest 1

6.tab test 2

7.T a b T e s t

9.first there are ordinary spaces, but now: a TAB

```
lsb@lsb-t61-mint ~ $
```

What it does is:

- first: grep every line with whitespace(s) in, and
- second: use sed on the grep output, to root out the lines, that do **not** have any TAB chars in them (in this case it removes only one line, number 8).

↩ ∞

jcxz100

Oct 28, 2014 @ 13:26

Got to correct myself, if this worked as expected:

```
$ grep '\t?' testgrep-tabs.txt
```

- it would match **every** line, as it asks for lines with “0-1 instances of a TAB char”

↩ ∞

PePa

Nov 14, 2015 @ 2:51

In bash, you can use `$'t'` to stand for a tab. So: `grep -c $'t'` (or `$'\n'` etc.)
GNU grep has a `-P` option (perl) that can be used: `grep -P 't'`

[↩](#) [∞](#)

flatcap

Feb 18, 2010 @ 10:34

Nice article.

One comment. You say:

> The following regex to find an IP address 192.168.1.254 will not work:
> `grep '192.168.1.254' /etc/hosts`

Actually, it *will* work; it will find the line you are looking for.
There's just a small chance of matching other things, too.

[↩](#) [∞](#)

Vance

Feb 19, 2010 @ 0:54

Tim:

You can do this with GNU grep also. For newlines, just use quotes before and after, e.g.

```
grep -c '  
' filename
```

(of course you can accomplish the same thing with

```
wc -l filename
```

:)

Tabs (and I assume formfeeds as well, though I haven't tested it) can also be entered at the command line. Type Ctrl-V before hitting tab and you'll get a literal tab instead of triggering filename autocompletion.

↩ ∞

Tim Boyer

Feb 19, 2010 @ 12:58

Vance –

The nl really isn't a problem, because, as you pointed out, there are other ways around it. Tabs are what I was shooting for, and your solution works perfectly! Thanks very much...

– tim –

↩ ∞

Jidifi

Feb 21, 2010 @ 15:26

Instead of:

```
egrep '[[[:digit:]]{1,3}\. [[[:digit:]]{1,3}\. [[[:digit:]]{1,3}\.
[[[:digit:]]{1,3} ' filename
```

I suggest:

```
egrep '([[:digit:]]{1,3}\.){3}[[:digit:]]{1,3} ' filename
```

or better:

```
egrep '([1-9][0-9]{0,2}\.){3}[1-9][0-9]{0,2}' filename
```

← ∞

Renjith KM

Oct 28, 2012 @ 16:40

valid IP address range is 0.0.0.0 to 255.255.255.255. So, I suggest the following:-

```
egrep '[0-255]{1,3}\.[0-255]{1,3}\.[0-255]{1,3}' my_file.txt
```

← ∞

Renjith KM

Oct 26, 2014 @ 10:44

```
egrep '[0-255]{1,3}\.[0-255]{1,3}\.[0-255]{1,3}\.[0-255]{1,3}' my_file.txt
```

← ∞

Shantanu Oak

Feb 23, 2010 @ 8:40

grep is very useful for analysing system resources. for e.g.

```
ps auxw | grep mysql
```

the tail -f command can be piped to grep like this...

```
tail -f /var/log/mysql-slow.log | grep 'someTable'
```

Show the 10 lines After and Before the selected word using -A 10 -B 10 -C 10 (for both, after and before)

Other useful switches are:

-r, -recursive

-l, -files-with-matches

← ∞

marcos

Apr 25, 2011 @ 19:28

Shantanu ,how can I get the line above of my search.

Thanks

← ∞

Shantanu Oak

Apr 26, 2011 @ 3:53

-B2 before context

Other useful options are:

-A2 after context

-C2 it will return 2 lines before and after context

for more:

man grep

← ∞

Ikem

Aug 29, 2010 @ 17:16

> How Do I do AND with grep?

>

> Use the following syntax to display all lines that contain both 'word1' and 'word2'

> \$ grep 'word1' _filename_ | grep 'word2'

← ∞

Anonymous

Sep 29, 2020 @ 16:32

This doesn't seem to work as i only get lines with word2 as output.

↩ ∞

Mr Plumb

Feb 17, 2011 @ 8:59

Thanks for the information.

However – why does the message at the top of the page have to keep changing? It means the text I am reading keeps bouncing up and down every few seconds, which is really annoying when you're trying to read it!

↩ ∞

nixCraft

Feb 17, 2011 @ 11:40

What are you talking about?

↩ ∞

roy

Feb 26, 2011 @ 22:10

How can I find all the rows that contain a certain string a given number of times?

↩ ∞

Mani

Apr 4, 2011 @ 19:53

How do i find a string using grep.

Say input file has

Vi_beaconen_h i_beaconen_h 0 PWL(

I want to print only " i_beaconen_h"

If i use

```
perl -lne '/ i/ and print' try.txt
```

It return whole line

If i use

```
grep -o 'i' try.txt
```

It returns only "i"

I want it to return "i_beaconen_h" [Or anything with i*]

I guess i m pretty new to perl and unix.

Mani !

↩ ∞

jack

Apr 4, 2014 @ 7:56

Not sure !!!

but try this one :

```
grep -o 'i_beaconen_h' file name
```

↩ ∞

suprabhat joshi

Apr 27, 2011 @ 8:39

how to display all lines the lines that have less than 9 character ?

↩ ∞

MikeW

May 11, 2011 @ 8:52

> how to display all lines the lines that have less than 9 character ?

Use the regexp feature below, with a preceding character expression

{n,m}

The preceding item is matched at least n times, but not more than m times.

eg.

`^\w\s]{0,8}$` will match rows of 0 to 8 word or space characters.

← ∞

nikhil

Nov 17, 2011 @ 6:49

Dear all,

I want to know how to grep an apache log file and save some details into a database, say like, somebody access a url like <http://site.com/test>. so in that i wanted to save the access url time and from which ip, only this three details i wanted to save in mysql database.

Thanks In advance.

← ∞

aaron

Feb 2, 2012 @ 4:52

How would I search a file and print 4-letter words that start and end with the letter a?

Count all words that contain the four letter sequence A, then two more letters, and then another A?

Count all words that contain a letter, two letters, and then a repeat of the first letter?

← ∞

Saurabh

Feb 15, 2012 @ 12:57

Thanks. Very useful information

↩ ∞

suruchi

Feb 17, 2012 @ 20:01

pls just hlp me out with this question..

↩ ∞

suruchi

Feb 17, 2012 @ 20:02

how will i Find all lines in a file with exactly 9 characters in them using grep command.

↩ ∞

Ashish Goyal

Feb 24, 2012 @ 4:55

grep '^.{9}\$' filename

↩ ∞

Smaran

Apr 9, 2012 @ 17:05

How do I find the occurrence of the following pattern

[x,y] (in the square brackets), where x and y are one or more digits.

Meaning if there is a pattern [8], it should not be displayed in the output

↩ ∞

Chris

May 8, 2012 @ 18:46

```
a='[12,111]'  
echo "$a" | grep "\[[0-9][0-9]*,[0-9][0-9]*\""
```

Had to do it this way in RHEL5 because of issues with some of the regular expressions. i.e. `echo "$a" | grep "\[[0-9]+,[0-9]+\]"` should work but doesn't and `echo "$a" | grep -e "\[[0-9]{1,}\,[0-9]{1,}\]"` should work but doesn't...

↩ ∞

Chris

May 8, 2012 @ 18:45

```
a='[12,111]'  
echo "$a" | grep "\[[0-9][0-9]*,[0-9][0-9]*\""
```

Had to do it this way in RHEL5 because of issues with some of the regular expressions. i.e. `echo "$a" | grep "\[[0-9]+,[0-9]+\]"` should work but doesn't and `echo "$a" | grep -e "\[[0-9]{1,}\,[0-9]{1,}\]"` should work but doesn't...

↩ ∞

Matt

Jun 27, 2012 @ 2:14

Hi Guys,

I'm just newbie with unix and is wondering if there's a way to grep a word in a vertical manner.

Example from a datafile:

a b c d e f g h
a b c g e f g h
a b c r e f g h
a b c e e f g h
a b c p e f g h
a b c d e f g h

On the third column from rows 2 to 5, the word 'grep' is formed vertically. Is there a way I can grep this or are there any other commands I could leverage?

Waiting for your expert advise :)

↩ ∞

Hossam Abd El-Monem

Jun 27, 2012 @ 11:01

Hi,

It would be done by the below ways:

```
cat word.txt | cut -d' ' -f4 | grep [g,r,e,p]
```

g
r
e
p

```
awk '{print $4;}' word.txt |grep [^d]
```

g
r
e
p

```
cat word.txt | tr d ' ' | cut -f4 -d\
```

g
r
e
p

↩ ∞

Sumit

Dec 26, 2012 @ 14:07

Hi,

I have to validate a String against a regular expression for a date format

'YYYYMMddhhmmss'. I have tested the below code,

```
temp=`echo $file_timestamp | egrep '^((20)[0-9][0-9]([01-9]|1[012])([01-9]|[12][0-9]|3[01])
(0[0-9]|1[0-9]|2[0123])([0-5])[0-9]([0-5])[0-9]$`';
```

The following returns the content of file_timestamp if it satisfies the pattern else returns null to the variable temp. If anyone can validate my understanding for the above snippet.

Thanks,

Sumit

↩ ∞

Chongee

Feb 24, 2013 @ 4:24

Hi guys

I have to export data from hundreds of output files, and all the output files contain this information based on some rules.

It's starting with ASM2_ , than sometimes comes BSSE_ sometimes don't, than every time comes one of these H3CO, BF3CO, BH3NH3, BF3NH3, BH3PH3, BH3, BF3, CO, NH3,

PH3 than _ than one of these HF, B3LYP, PW91 than / and than one of these 6-31G(d), 6-311G(d), 6-311++G(2d,p) and this is the end of line.

One example would be

ASM2_BH3CO_HF/6-311++G(2d,p)

The problem is that these things will appear many times alone in the text, but just once in this order and as one line from start to end.

Can I do something about it with grep, or I would have to use something else?

Thx

← ∞

Jason Chia

Apr 14, 2013 @ 16:35

Hi, does anyone know how I can use grep to only show word matches that start with c for example? Cuz I was thinking of using the wildcard "c*" but that wouldn't work in grep since it uses regex which has a different meaning for *. So what I want to ask is: What is the regex equivalent of "c*"? Thanks in advance.

← ∞

answer

May 21, 2013 @ 14:33

Jason, you can use the "word boundary" expression, which depending on what tool you're using can be either \b or \<

* is a quantifier, so "c*" would match "zero, one or more 'c' characters". You need exactly one c followed by anything, that would be:

\bc.*

← ∞

John

Jun 17, 2013 @ 13:03

Hi Vivek,

I am working on analysis of one of the website and I am using grep command.

Our basic requirement is:

1. it has to start with upper case or lower case letter.
2. it has to be more than 4 characters.
3. it should end with following punctuations: .,!?

So basically we are looking about 10000 files. we pick particular extension of file and search for that file through out the directories and then try to find all english sentences in these source files(exmp. .java, .jsp, .html, .js etc). Trying to filter coding part basically ..and count how many matches we found ...

I used following commands to check but there are not giving 100% result :

```
$ find -name "*.html"| xargs grep -e ^[A-Za-z]\{4\} -e '[.,!?!?]'$ for java
```

```
$ find -name "*.html"| xargs grep -e ^[A-Za-z]\{4\}.*[.,!?!?]"$ for html
```

Can you please let me know what am i doing wrong?

I appreciate for all your help.

Thanks in advance ..

← ∞

Vikram H S

Jul 30, 2013 @ 7:23

Hi,

Would be glad if anyone could help me out.

I have recieved a file which cotains unknown character,below are few characters

Â·Ã¡

I'm using a grep command to find if the character is present, followed by sed to replace these character with "(empty space).

I'm worried if i receive any other unknown characters.

Thanks in advance,
Vikram

↩ ∞

wiske57

Aug 23, 2013 @ 23:43

Very well done! Thanks.

↩ ∞

Maxim

Sep 6, 2013 @ 6:09

grep ".*test1.*test2.*test3" filename

this can find lines in file which contain test1,test2 and test3 patterns

↩ ∞

Sekar

Nov 22, 2013 @ 11:10

Hi

i need to find the lines which is not only contain the specific pattern....

EX: need to find the lines not only contain [A-Z]?????

↩ ∞

Ryan

Feb 26, 2014 @ 19:51

grep -E '[^A-Z]'

↩ ∞

Ella

Jan 29, 2014 @ 16:04

Hi,

Is there any possibility to grep for series of numbers in single command ?

Eg:

E140

ED41

EF42

EA43

From the above have to grep for sequence of numbers [40-43]

Please could someone suggest?

↩ ∞

Ryan

Feb 26, 2014 @ 19:58

For your particular case:

grep -E '4[0-3]' a.txt

Though it is limited to a 10 digit range as you can see.

↩ ∞

bill

Jan 30, 2014 @ 18:08

hi,

Is there a way to grep for the line which end with a space?

Thanks for any suggestions!

↩ ∞

Ryan

Feb 26, 2014 @ 19:48

grep -E ' \$' filename

↩ ∞

Kamran

Mar 9, 2014 @ 3:23

@

Eg:

E140

ED41

EF42

EA43

More likely 'cut' will do it –

cut -c 3,4 filename

↩ ∞

eSensible

May 22, 2014 @ 13:03

“grep -irl”, what a great nerd pun

↩ ∞

Haritha

Sep 24, 2014 @ 4:41

Hi,

I am wondering if there is a way I can do this with egrep:

I want to specify a pattern as something that does not contain a set of given patterns.

I am trying to find if the text has patterns of the form `u"..."` where `,` and should not contain `"` or `,` `u'`. If they contain either `"` or `,` `u'` then I am not interested in that pattern.

If I find such a pattern, I want to replace it as `u"`

How to do this using unix tools. Can I write a shell script to do this?

 

Haritha

Sep 24, 2014 @ 4:42

Hi,

I am wondering if there is a way I can do this with egrep:

I want to specify a pattern as something that does not contain a set of given patterns.

I am trying to find if the text has patterns of the form `u"part1"part2 "part3"` where `part1,part2` and `part3` should not contain `"` or `,` `u'`

If I find such a pattern, I want to replace it as `u"part1part2part3"`

How to do this using unix tools. Can I write a shell script to do this?

shah

Sep 24, 2014 @ 16:24

Please can you be more precise of your problem , just post the text for which you want to have a pattern.

So far i can understand first part of your question , for that solution is to use either " ^ " or -v with the grep.

Searching for multiple patterns , egrep is the way to do it . Reg exp are always in single quotes while a string in double quotes. Not to be ignored , Reg exp just means strings with wildcards or special characters.

Search & replace can be best performed in three ways –

1 – sed

2 – tr

3 – vi editor

As far as i'm concerned there 're hundreds of other way to go from A to B in unix ,but these were the simplest i could think of.

Haritha

Sep 27, 2014 @ 3:34

When i search for a pattern like `u"[^"]*"["^"]*"["^"]*"`

It can give me this :

`u"somethinghere", u'somethinghere' : u"somethinghere"`

But what I really want to check is if the text has patterns like :
u"somethinghere"somethinghere"somethinghere" where none of the somethinghere has
a " or , u' in it. This mean the pattern for somethinghere is like: should not contain
double quote or the character sequence , u'

I hope this is more clear. Thank you for the prompt response.

↩ ∞

Shah

Oct 9, 2014 @ 0:44

Look ,

" " – double quotes means string

' ' – single quotes means regular exp or pattern or strings with wildcards(special char)

example –

Searching for this – myipadd192.168.0.1

egrep '[0-255]\.' /dir/filename

If you gotta look for pattern , forget about the text attached to it. Just go for the pattern
.Also not to miss diff between grep & egrep.

↩ ∞

Linux Lewis

Dec 18, 2014 @ 3:18

Wow, this is insanely helpful. regex is seriously covered on the LX0-101 exam, but you
won't find anything on it with the LabSim or Skillsoft courses. You have to dig for it.
Nslookup won't do here.

↩ ∞

Sukaina

Dec 30, 2014 @ 17:31

Hi,

I need to grep from a big 6GB oacle alert.log file.

Issue is that the date is on one line then the related matter below it, e.g.:

Mon Dec 29 02:26:06 2014

MTTR advisory is disabled because FAST_START_MTTR_TARGET is not set

Mon Dec 29 02:26:06 2014

SMON: enabling cache recovery

Mon Dec 29 02:26:06 2014

NOTE: dependency between database b1almpp and diskgroup resource ora.DATA.dg is established

Tue Dec 30 02:25:25 2014

GTX0 started with pid=51, OS id=15088

Starting background process RCBG

Tue Dec 30 02:25:25 2014

RCBG started with pid=52, OS id=15092

replication_dependency_tracking turned off (no async multimaster replication found)

Starting background process AQPC

Tue Dec 30 02:25:26 2014

AQPC started with pid=54, OS id=15112

Starting background process CJQ0

Completed: ALTER DATABASE OPEN /* db agent */ {1:26602:59235} */

and so on.

i want to grep the date e.g. Dec 30, but i am getting only that line not the lines below it, i need the lines below the date lines too

i am giving ->

grep -i "Tue Dec 30 0*" alert.log

result i get is:

Tue Dec 30 02:25:25 2014

Tue Dec 30 02:25:25 2014

Tue Dec 30 02:25:25 2014

Tue Dec 30 02:25:25 2014

Tue Dec 30 02:25:25 2014

Tue Dec 30 02:25:25 2014

Tue Dec 30 02:25:25 2014

i want result as:

Tue Dec 30 02:25:25 2014

SMCO started with pid=48, OS id=15074

Tue Dec 30 02:25:25 2014

minact-scn: Inst 1 is now the master inc#:4 mmon proc-id:14890 status:0x7

minact-scn status: grec-scn:0x0000.00000000 gmin-scn:0x0000.00000000 gcalc-scn:0x0000.00000000

minact-scn: Master returning as live inst:2 has inc# mismatch instinc:0 cur:4 errcnt:0

Tue Dec 30 02:25:25 2014

Opening with Resource Manager plan: DEFAULT_PLAN

Starting background process GTX0

Tue Dec 30 02:25:25 2014

GTX0 started with pid=51, OS id=15088

please help me at earliest.

Thanks,

Sukaina

↩ ∞

justFrustrated

Jan 20, 2015 @ 14:30

I am trying to comment all the citations in a tex file in a directory.

this should work...

```
find . -name "*.tex" -print | xargs sed -ri 's/~\cite{*/%~\cite{*}\n/g'
```

so all the citations are replaced by the same expression only with % in front and a new line at the end so ~\cite{blah} becomes

%~\cite{blah}

but is not only does not throw any error, it does nothing at all

← ∞

Steve

Feb 7, 2015 @ 6:47

Great write up.

Thank you.

← ∞

tod

May 22, 2015 @ 9:25

clear, concise, useful. nixCraft is the best

← ∞

darek

Apr 28, 2016 @ 9:53

Well described. Thank you.

Eric Van Buggenhaut

Jul 18, 2016 @ 19:35

character classes seem to be declared with double brackets now:

```
$ grep -r '[:space:]+$' Controller/*
```

grep: character class syntax is `[[:space:]]`, not `[:space:]`

John

Jul 22, 2016 @ 3:35

Hi,

What is the significance of `.\+` in grep?

The dot (`.`) is for matching any single character and `\+` is for numerous matching instances of preceding character.

A file containing 345362, 23423 on separate lines returns all the lines when `.\+` is used. If first 3 was matched, it should be matching subsequent 3's and not other characters such as 345362.

Please help.

richard

Mar 21, 2017 @ 14:37

hello can some one please help me i would like to grep a password from an html file on my own network but i cannot get my head round how to only display the correct bit of info.

here is the example of the file

")

Username: **blahhhhhh**

Password: **this is what i need and want**

"

and my grep command of this

"egrep -o 'Password: *.*\$'"

gives me the entire line

↩ ∞

sudhakar

Sep 12, 2017 @ 10:45

Sir, you have saved my day

↩ ∞

Piotrek

Aug 15, 2020 @ 6:37

A quick refresher doing the job :-)

↩ ∞

aditi

Aug 27, 2020 @ 4:38

Display whose 3rd character of name is r

Display whose name ending with e or i

↩ ∞

Saanvi

Feb 20, 2022 @ 7:27

The 2nd one:

```
echo 'apple' | egrep -o '(e|i)$'
```

↩ ∞

Ted

Dec 10, 2020 @ 11:10

Thanks a lot !

Brilliant, very useful !

↩ ∞

Paul Wratt

Aug 7, 2022 @ 10:58

my search results said this page was updated on the 23 April 2022

nixcraft + cyberciti.biz is the best

the only legitimate .biz url that ever existed

I hope you allow archive dot org crawling, as I would hate to live in a world without the entire content of cyberciti.biz , even in 50 years

thnx again nixcraft

↩ ∞

Illia

Sep 27, 2022 @ 21:06

Thank you a lot – helped me solve the issues in no time! Keep going!

← ∞

Busch

Jun 17, 2023 @ 21:14

Hi,

Excellent and helpful pages for new developers like me. Can you add pdf support for these tutorials?

← ∞

The Andy Narlenson

Sep 6, 2023 @ 11:03

Very helpful indeed! The most documented and exact article I've read about grep! Thanks for making this kind of tutorials instead of short & clickbaity stuff <3

← ∞

Use HTML `<pre>...</pre>` for code samples. Your comment will appear only after approval by the site admin.

Next FAQ: [Linux tar Extract Files Command](#)

Previous FAQ: [HowTo: Linux Rename Files](#)